

O Pensamento Computacional como Ferramenta de Apoio ao Raciocínio Matemático

1

Abstract. *The shift from arithmetic to algebra requires generalizing discrete values into abstract variables. Computational Thinking provides an applied environment to test these concepts. Based on Papert (1980) and Weintrop et al. (2016), this paper investigates the practical intersections between algorithm design and Mathematical Logic. We examine parametric decomposition and propose modeling Monte Carlo simulations in Python to teach Probability. By programming uniform distributions to approximate Pi, students actively map conditional statements to logical axioms.*

Resumo. *A transição da aritmética para a álgebra exige extrapolar o cálculo discreto para abstrações. O Pensamento Computacional fornece um ambiente aplicado para testar essas generalizações. Fundamentado em Papert (1980) e Weintrop et al. (2016), este artigo investiga as interseções práticas entre algoritmos e Lógica Matemática. Examinamos a decomposição paramétrica e propomos modelar simulações de Monte Carlo em Python no ensino de Probabilidade. Ao programar distribuições para aproximar Pi, o aluno mapeia estruturas condicionais em axiomas lógicos.*

1. Introdução

O ensino tradicional de Álgebra frequentemente falha ao exigir que o aluno compreenda variáveis sem um suporte concreto para manipulá-las. Nesse cenário, o Pensamento Computacional atua como uma linguagem mediadora. Ao implementar um laço como `for i in range(n):` em Python, o termo `n` atua operacionalmente como o limite superior de uma iteração finita. Em vez de ser apenas um símbolo estático no quadro, `n` determina a alocação de memória e a quantidade exata de ciclos que o processador executará, permitindo ao aluno observar diretamente como a alteração desse parâmetro redimensiona a execução do algoritmo.

Apoiando-se nas Práticas de Modelagem e Simulação descritas por [Weintrop et al. 2016], este artigo explora como a construção de algoritmos exige a aplicação funcional da Lógica Matemática. A criação de um programa de computador requer o uso estrito de conectivos lógicos, como as conjunções e disjunções presentes nas estruturas de controle de fluxo (condicionais *if/else*). Com isso, a formulação de proposições passa a compor a mecânica de execução do código, exigindo que o aluno aplique condições consistentes para que o programa produza a saída correta.

Para ilustrar essa abordagem, aplicamos a decomposição paramétrica no ensino de Probabilidade. Essa decomposição ocorre quando o estudante isola os componentes de um sistema lógico — por exemplo, separando o raio de um círculo (uma constante

geométrica) da geração de coordenadas aleatórias (variáveis de estado). Metodologicamente, propomos o desenvolvimento de simulações do método de Monte Carlo em Python. Utilizando a função `random.uniform()`, o aluno gera uma distribuição uniforme de coordenadas (x, y) e aplica a métrica de distância euclidiana para determinar se o ponto gerado pertence à área do círculo. Essa modelagem submete a lógica geométrica a milhões de ciclos iterativos, comprovando empiricamente a convergência probabilística estabelecida pela Lei dos Grandes Números.

2. Fundamentação Teórica

2.1. Abstração e decomposição matemática

O Pensamento Computacional exige a desconstrução do problema. A decomposição, sob a ótica matemática, ultrapassa a simples divisão de tarefas. Ao resolver o cálculo de volume de sólidos pelo método de fatiamento, por exemplo, a decomposição algorítmica exige que o estudante identifique os parâmetros invariáveis (como a altura total) e os separe das variáveis iterativas (como a espessura de cada fatia infinitesimal), estruturando um laço de repetição acumulador.

Paralelamente, a abstração computacional serve como apoio direto para a abstração algébrica. Ao transformar um valor fixo em uma variável paramétrica no código, o estudante vivencia a transição cognitiva necessária para compreender o papel da variável x na Álgebra. O ambiente de programação exige precisão na formulação das regras, fornecendo *feedback* imediato caso a premissa lógica esteja incorreta.

2.2. Modelagem e construcionismo

A exigência de converter ideias em código executável requer precisão operacional. Diferente da resolução tradicional no caderno, o ambiente de desenvolvimento acusa falhas lógicas instantaneamente. Essa dinâmica corrobora os achados de [Ng e Cui 2021], que demonstraram como a criação de algoritmos impulsiona a revisão contínua do próprio pensamento matemático do aluno.

Essa abordagem encontra sustentação no Construcionismo de [Papert 1980]. Nessa ótica, a tecnologia atinge seu real potencial educacional quando o estudante passa a programar o computador, e não o inverso. Ao codificar, ele arquiteta seu próprio modelo cognitivo e valida os passos da resolução de problemas matemáticos por meio da execução de seus scripts.

3. Metodologia

Este estudo caracteriza-se como pesquisa bibliográfica exploratória de abordagem qualitativa, fundamentada nas categorias de análise propostas por [Weintrop et al. 2016]. Articulando referenciais teóricos de educação com fundamentos de engenharia de software, o trabalho propõe um modelo aplicado baseado na criação de simulações estocásticas em Python. A adoção dessa linguagem justifica-se por sua legibilidade estrutural, permitindo codificar conceitos de Probabilidade em artefatos que o aluno pode auditar iterativamente.

4. Aplicações Pedagógicas

4.1. Aprendizagem Baseada em Problemas (ABP)

Ao adotar a ABP integrada à computação, o professor orienta o desenvolvimento de algoritmos para solucionar desafios aplicados, como modelagem de rotas logísticas. Con-

forme destacam [Rezende e Silva-Salse 2021], a ABP aliada ao Pensamento Computacional transforma a disciplina de matemática em um processo focado em investigação e design.

4.2. Simulação de Monte Carlo

No ensino de Probabilidade, o estudante deve atuar como arquiteto do modelo estocástico [Wilensky 1995]. A implementação do Método de Monte Carlo materializa essa teoria. Ao codificar a aproximação geométrica de Pi, o aluno define variáveis para simular pontos no espaço cartesiano.

```
import random

def aproximar_pi_monte_carlo(pontos_totais):
    pontos_no_circulo = 0
    for _ in range(pontos_totais):
        # Distribuição uniforme de coordenadas x e y entre 0 e 1
        x, y = random.uniform(0, 1), random.uniform(0, 1)

        # Condicional baseada na distância euclidiana ( $x^2 + y^2 \leq r^2$ )
        if (x**2 + y**2) <= 1:
            pontos_no_circulo += 1

    # Cálculo final das métricas de aproximação
    return 4 * pontos_no_circulo / pontos_totais
```

Neste trecho de código, a métrica de distância euclidiana é avaliada na instrução `if`. Se a condição for verdadeira, o algoritmo computa o evento favorável. Isso consolida o cruzamento entre Álgebra, Geometria e Probabilidade em uma única estrutura executável.

4.3. Depuração (debugging)

A prática de depuração (*debugging*) altera a percepção do erro, tratando-o como etapa de refinamento lógico. Erros de compilação ou de resultados forçam o aluno a rastrear e inspecionar o fluxo de execução, diferenciando desvios sintáticos da linguagem das falhas de formulação matemática.

5. Resultados e Impactos Esperados

A Tabela 1 sintetiza os impactos esperados na transição de um modelo educacional puramente tradicional para um modelo integrado ao Pensamento Computacional.

6. Considerações Finais

A inserção do Pensamento Computacional consolida-se como uma estrutura pedagógica viável para a execução prática da Lógica Matemática. A criação de código oferece um ambiente onde as abstrações deixam de ser simbologias vazias e ganham utilidade programável. Alinhada às recomendações de [Weintrop et al. 2016], a disrupção metodológica proposta está em posicionar o estudante na condição de projetista de regras, permitindo que conceitos estocásticos passem a compor a resolução iterativa e testável de problemas.

Tabela 1. Diferenças paradigmáticas entre a abordagem tradicional e a instrução orientada ao Pensamento Computacional

Dimensão Cognitiva	Ensino Matemático Tradicional	Apoiada pelo PC
Problemas	Foco na decoreba de fórmulas e substituição de valores para cálculos fechados.	Decomposição sistêmica e abstração paramétrica para modelagem iterativa.
Tratamento do Erro	Visto como fracasso de raciocínio, resultando em dedução imediata de notas.	Encarado como evento de depuração lógica que exige auditoria e revisão do modelo.
Escalabilidade	Limita-se a amostras muito pequenas calculadas manualmente no papel.	Capacidade de processar milhões de iterações em segundos (ex: simulação de Monte Carlo).

Referências

- Barcelos, T., Muñoz, R., Villarroel, R. A., e Silveira, I. F. (2015). Relações entre o pensamento computacional e a matemática: uma revisão sistemática da literatura. In *Anais dos 4º Workshops do Congresso Brasileiro de Informática na Educação*, páginas 1369–1378, Maceió. SBC.
- Brasil. Ministério da Educação (2018). *Base Nacional Comum Curricular*. MEC, Brasília, DF.
- Chan, S.-W., Looi, C.-K., Ho, W. K., e Kim, M. S. (2022). Tools and approaches for integrating computational thinking and mathematics: a scoping review of current empirical studies. *Journal of Educational Computing Research*, 60(8):2036–2080.
- Ng, O.-L. e Cui, Z. (2021). Examining primary students’ mathematical problem-solving in a programming context: towards computationally enhanced mathematics education. *ZDM – Mathematics Education*, 53:847–860.
- Papert, S. (1980). *Mindstorms: children, computers, and powerful ideas*. Basic Books, New York.
- Rezende, A. A. e Silva-Salse, A. R. (2021). Utilização da aprendizagem baseada em problemas (ABP) para o desenvolvimento do pensamento crítico (PC) em Matemática: uma revisão teórica. *Educação Matemática Debate*, 5(11):1–21.
- Weintrop, D., Beheshti, E., Horn, M., Orton, K., Jugo, K., Trout, J., e Wilensky, U. (2016). Defining computational thinking for mathematics and science classrooms. *Journal of Science Education and Technology*, 25(1):127–147.
- Wilensky, U. (1995). Learning probability through building computational models. In *Proceedings of the 19th International Conference on the Psychology of Mathematics Education*, Recife. PME.
- Wing, J. M. (2006). Computational thinking. *Communications of the ACM*, 49(3):33–35.
- Wisniewski, B., Zierer, K., e Hattie, J. (2020). The power of feedback revisited: a meta-analysis of educational feedback research. *Frontiers in Psychology*, 10:1–14.